`std::array` is a wrapper for an array!

Document number:
P3737R0

Date:
2025-06-08

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3737/github

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/array.cow

ABSTRACT: The `std::array` class template is implemented as a simple wrapper type for a "C-style array". However, its specification in the standard is considerably more permissive and should be simplified.

§ Contents

§ 1. Introduction

The `std::array` class template has established itself as a de-facto replacement for "builtin arrays" or "C-style arrays" in many code bases. This also means that it is frequently taught to novice programmers, with an explanation along the lines of:

> `std::array` is just a wrapper for a C-style array:
>
> ```
> template <typename T, size_t N>
> struct array {
>     T __array[N];
>     // ...
> }
> ```

While this explanation is not correct for zero-length `std::arrays`, it does match how the template is implemented in every standard library for `N != 0`, and there is very little reason not to implemented it in this obvious fashion.

§ 1.1. What the standard says

The actual specification of `std::array` is not so simple, and is a combination of multiple constraints on the implementation:

- It is a class template, a contiguous container ([array.overview] paragraph 1), and a reversible container (with an exception; see [array.overview] paragraph 3), and it meets some requirements of a sequence container.
- It can be list-initialized with up to N elements whose types are convertible to `T` ([array.overview] paragraph 2). This is obviously not exhaustive; initialization with `{}` or `{other_array}` should also be possible.
- It is a structural type if `T` is a structural type ([array.overview] paragraph 4), and therefore, also a literal class type in that event.

Additionally, while this does not strictly specify anything about the layout, some helper functions in the standard library de-facto rely on it. Take [array.creation] for example:

> ```
> template<class T, size_t N>
>   constexpr array<remove_cv_t<T>, N> to_array(T (&a)[N]);
> ```
>
> *Mandates*: `is_array_v<T>` is `false` and `is_constructible_v<remove_cv_t<T>, T&>` is `true`.
>
> *Preconditions*: `T` meets the *Cpp17CopyConstructible* requirements.
>
> *Returns*: `{{ a[0], …, a[N - 1] }}`

The use of double braces in the *Returns* specification would be nonsensical if `std::array` was not "a wrapper for an array".

§ 1.2. What the standard does not say

Notably, there are quite a couple of guarantees that are absent.

EXAMPLE: It would be compliant to implement `std::array` as follows:

```
struct alignas(1024) malice_and_evil {
    constexpr malice_and_evil() { }
    malice_and_evil(const malice_and_evil&) { }
};

template <typename T, size_t N>
struct array {
    T __array[N];
    malice_and_evil evil;
};
```

Such an implementation technically satisfies all the requirements for `std::array`, but

- copying the array would not be a constant expression,
- `std::array` would not be trivially copyable for any type, and
- its size and alignment would be much greater than that of `T[N]`.

EXAMPLE: An even more insane implementation would be:

```
struct gobbler {
    constexpr gobbler() = default;
    constexpr gobbler(auto&&) {}
};

template <typename T, size_t N>
struct array {
```

```
    gobbler __gobblers[N];
    T __array[N]; // necessary to satisfy contiguous container requirements
    // ...
};
```

Since [array] never states what effect list-initialization has for a `std::array`, and even `std::to_array` is just stated to return the result of *some expression*, nothing suggests that `begin()` would give us an iterator to `x` after initializing like `std::array<T,1>{{x}}`. All list-initialization could be "gobbled up".

§ 2. Motivation

It seems like the vagueness in the specification serves no practical purpose; it is unclear what implementations could do with the additional freedom, other than pranking their users. It would be beneficial to the C++ community if the simplified explanation in §1. Introduction was what the standard actually said.

A stricter specification would provide additional useful guarantees such as `std::array<T, N>` being trivially copyable when `T` is trivially copyable. This is relevant to use cases like `std::bit_cast<std::array<std::byte, sizeof(x)>>(x)`, which *technically* rely on implementation details, not on standard behavior.

§ 2.1. Isn't this a waste of time?

While it could be argued that only a malicious implementation would violate our user expectations as in §1.2. What the standard does not say and it is therefore time-wasting to restrict `std::array` any further, it would be unusual for WG21 to shy away from standardizing universally existing practice and to recommend users to rely on non-standard implementation details, simply because those implementation details are widespread. If the remaining implementation freedom can only be used for evil, perhaps we should not grant it.

§ 3. Design considerations

While the specification for arrays of nonzero length is rather obvious, it is unclear how many guarantees we want to provide for zero-length arrays. For example, should `std::array<std::string, 0>` be trivially copyable, even though `std::string` is not?

Within [array.zero], there are some long-standing issues going back to 2012. LWG has visited this subclause many times in [LWG2157], but never fully completed a solution. This work has been absorbed mostly unmodified into §5. Wording.

§ 3.1. Zero-length `std::array` status quo

The zero-length case is also where we see some implementation divergence in size and alignment of the array. The following table shows how major standard libraries implement zero-length `std::array`.

Library	Implementation	Size	Trivially copyable	Assignable
MSVC STL	contains `T` if `T` is default-constructible, otherwise `struct{}`	`sizeof(T)` or `1`	depends on `T`	depends on `T`
libstdc++	contains `struct{}`	`1`	always	always
libc++	contains (possibly const) `__empty[sizeof(T)]`	`sizeof(T)`	always	depends on `T`

BUG: The MSVC STL implementation is non-compliant and insane. Despite `std::array<T,0>` being a *zero-length* container with *no* elements, it will actually hold one element (and call its constructors and destructors) as long as `T` is default-constructible.

3.1.1. Conclusion

Generally speaking, it is desirable if a zero-length `std::array` behaves as similarly to a regular `std::array` of the same element type. libc++ is the only implementation that does this well. The "greatest common denominator" between these implementations should be standardized, which is:

- `std::array<T, 0>` is trivially copyable if `T` is.
- `std::array<T, 0>` is assignable if `T` is.
- `std::array<T, 0>` has size and alignment at most that of `T`.
- `std::array<T, 0>` is not an empty class.

Without breaking ABI, that implementation would look something like:

```
struct __empty {};

template<class T>
struct array<T, 0> {
  // const __empty if T is const
  using __empty_type = copy-cv<T, __empty>;

  // No alignas for libstdc++.
  alignas(T) __empty_type arr;
};
```

3.2. Trivial copyability of zero-length arrays

Whether a type is trivially copyable has ABI impact. It may change whether the type is passed via register or one the stack, and so we cannot mandate any change to this behavior without

breaking ABI.

For MSVC, a `std::array<int,0>` is trivially copyable, but a `std::array<std::string,0>` is not. This seems reasonable; there is no strong motivation for making arrays trivially copyable even if a nonzero variant of the same array wouldn't have been. In fact, it could be argued that this is surprising and inconsistent.

3.2.1. Conclusion

Mandate that `std::array<T,0>` is trivially copyable if `T` is. Otherwise leave this up to implementations.

3.3. Double-brace initialization for zero-length arrays

Note that we need to make double-brace initialization like `std::array<int, 0>{{}}` valid to make generic programming easier. It is plausible that we perform this when expanding an empty pack like: `std::array<int, sizeof...(args)>{{ args... }}`.

Making this valid requires either some non-static data member, or a base class. An empty base class would make the array as a whole an empty class, and this would be an ABI break, so it is out of the question.

3.3.1. Conclusion

Standardize the existing practice of having a non-static data member which enables double-brace initialization.

3.4. Problematic iterator requirements for zero-length arrays

[array.zero] paragraph 2 specifies:

> In the case that `N == 0`, `begin() == end()` == unique value. The return value of `data()` is unspecified.

Firstly, it is unclear whether this "unique value" is meant to be unique per object, unique for each invocation, etc.

Secondly, this requirement was never implemented by any compiler and it is too late to fix now. Note that MSVC STL, libc++, and libstdc++ all use `T*` as an iterator type. Considering that, a possible implementation looks like:

```
template<class T>
struct zero_length_array {
  union U {
    char c;
    int i;
    U() = default;
  } u;
```

```
  constexpr const T* begin() const noexcept { return &u.i; }
  constexpr const T* end() const noexcept { return begin(); }
  // ...
};

constexpr zero_length_array<int> a{{}}; // OK
static_assert(a.begin() == a.end());    // OK
```

However, this would require `std::array<T,0>` to be at least one `T` large, and it is only a single byte large for libstdc++. Changing the size of the type would break ABI. The only way to conjure up a `T*` out of thin air would be to use `reinterpret_cast`, but that would not work in constant expressions.

3.4.1. Conclusion

Delete the uniqueness requirement. Without specifying anything special for zero-length arrays, it still acts as an empty range, and `begin() == end()` is `true`, which is all we really need.

3.5. `std::array<T,0>::front()` is `std::unreachable()`?!

`std::array<T, 0>::front()` is entirely undefined, making it equivalent to `std::unreachable()`, which feels out-of-place especially following C++26, where `front()` normally has a *Hardened preconditions* specification.

Note that deleting `front()` and `back()` for zero-length arrays is not feasible because of (existing) code along the lines of:

```
if (a.size() != 0) {
    return a.front();
}
```

It is quite plausible that `std::array<T,0>::front()` is called in code that is logically unreachable, so it should not result in a compiler error, which would be the consequence of `= delete;`.

3.5.1. Conclusion

Make `front()` and `back()` always result in a contract violation, as if they had *Hardened preconditions* that are always violated. However, if the implementation is not hardened, these functions should simply terminate instead of being another spelling for `std::unreachable()`.

4. Impact on implementations

Existing implementations of `std::array` are virtually unaffected. One exception to this is that the behavior of `std::array<T, 0>::front()` is no longer undefined; see below for specifics.

5. Wording

The following changes are relative to [N5008].

5.1. [array.overview]

Change [array.overview] paragraph 1 as follows:

> The header `<array>` defines a class template for storing fixed-size sequences of objects. ~~An array is a contiguous container ([container.reqmts]).~~ An ~~instance~~ **object** of **type** `array<T, N>` stores `N` elements of type `T`, so that ~~`size() == N` is an invariant~~ **`size()` always equals `N`**.

Change [array.overview] paragraph 2 as follows:

> An `array` is an aggregate ([dcl.init.aggr]) ~~that can be list-initialized with up to `N` elements whose types are convertible to `T`~~ **with no base classes. A specialization `array<T, N>` has a single public non-static data member of type "array of `N` `T`" if `N` is nonzero; otherwise the contents are specified in [array.zero]**.
>
> **[*Note*: An `array` is trivially copyable, standard-layout, and a structural type if `T` is trivially copyable, standard-layout, and a structural type, respectively. — *end note*]**

Change [array.overview] paragraph 3 as follows:

> An `array` meets all of the requirements of a container ([container.reqmts]), **of a contiguous container,** and of a reversible container ([container.rev.reqmts]), except that a ~~default constructed `array` object~~ **default-initialized or value-initialized object of type `array<T, N>`** is not empty if `N` > 0. An `array` meets some of the requirements of a sequence container ([sequence.reqmts]). Descriptions are provided here only for operations on `array` that are not described in one of these tables, and for operations where there is additional semantic information.

Delete [array.overview] paragraph 4:

> ~~`array<T, N>` is a structural type ([term.structural.type]) if `T` is a structural type. Two values `a1` and `a2` of type `array<T, N>` are template-argument-equivalent ([temp.type]) if and only if each pair of corresponding elements in `a1` and `a2` are template-argument-equivalent.~~

Change [array.overview] paragraph 5 as follows:

```
namespace std {
  template<class T, size_t N>
  struct array {
    // non-static data members
```

```
    T arr[N];                                // name is exposition-only

    // types
    using value_type              = T;
    using pointer                 = T*;
    […]
  };
}
```

§ 5.2. [array.zero]

Delete all paragraphs within [array.zero]:

1 `array` shall provide support for the special case `N == 0`.

2 In the case that `N == 0`, `begin() == end() ==` unique value. The return value of `data()` is unspecified.

3 The effect of calling `front()` or `back()` for a zero-sized array is undefined.

4 Member function `swap()` shall have a non-throwing exception specification.

Insert new paragraphs within [array.zero]:

1 A specialization `array<T, 0>` does not have an *arr* data member. Instead, it has a non-static data member of unspecified, trivially copyable, standard-layout, empty aggregate type `U` with no base classes and with the same cv-qualification as `T`. The size and alignment of `U` is an implementation-defined choice between `1` and the size and alignment of `T`.

2 The value representation of an `array<T, 0>` is empty.

3 The `begin`, `end`, `cbegin`, `cend`, `rbegin`, `rend`, and `data` member functions of an `array<T, 0>` return value-initialized results.

4 The `fill` and `swap` member functions of an `array<T, 0>` are equivalent to functions with a *function-body* `{}`, and have a non-throwing exception specification.

5 `std::terminate` is invoked when execution reaches the end of member functions `operator[]`, `front`, or `back` of an `array<T, 0>`.
[*Example*: This can occur if the implementation is not hardened, or if the contract violation ([basic.contract.eval]) resulting from the function call is evaluated with ignore semantic. — *end example*]

§ 6. References

[N5008] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-03-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf

[LWG2157] *Daryle Walker.* How does std::array<T,0> initialization work when T is not default-constructible? (2012-05-08)
https://cplusplus.github.io/LWG/issue2157